

클로봇 합격 대비 AIP 면접 전략서

공부하면서 면접 답변을 만드는 프로젝트 중심 복습 자료



구분	내용
목적	클로봇 로봇 응용 SW 개발자 공고에 맞춰 AIP 프로젝트를 면접 답변으로 정리
학습 흐름	공고 분석 -> 프로젝트 소개 -> ROS2/웹관제/제어/센서 -> 시뮬레이션 -> 실차 확장 -> 예상 질문
작성 원칙	코드와 문서로 확인된 범위만 '확인됨'으로 표현하고, 실차 3대 완전 자율주행은 확인 필요로 분리
현재 데모	시뮬레이션 웹관제 http://localhost:18080 , Foxglove ws://localhost:18765
기준일	2026-07-06

반드시 기억	- AIP는 ROS2 Topic 계약으로 여러 차량의 상태와 제어를 묶는 관제 프로젝트다. - 현재는 시뮬레이션과 웹관제 검증이 중심이고, 실차 3대 완전 자율주행은 검증 예정이다. - 면접에서는 구현 범위와 확인 필요 범위를 분리해서 말해야 한다.
면접 한 문장	이 프로젝트는 로봇 상태, 제어, 센서 데이터를 ROS2와 웹관제로 연결한 신입 포트폴리오입니다.
주의	실차 3대 자율주행 완료, 상용 수준 안정화 완료처럼 근거가 없는 표현은 쓰지 않는다.

0. 이 PDF를 공부하는 방법

회독	목표	하는 일
1회독	구조 잡기	각 장의 '왜 필요한가'와 데이터 흐름을 먼저 읽고 전체 시스템 그림을 만든다.
2회독	답변 만들기	'면접에서 어떻게 말할까'를 본인 말투로 30초 답변으로 줄인다.
3회독	암기와 방어	'반드시 기억' 3줄과 '주의'를 외워 과장 질문에 흔들리지 않는다.
실습	말로 설명	상태 흐름, 제어 흐름, 실차 확장 전략을 종이에 직접 그려본다.

면접 준비의 핵심

클로봇 공고는 ROS2, 웹-로봇 연동, 실시간 데이터 시각화, 센서 연동, 테스트와 배포를 모두 본다. 그래서 단순히 '웹을 만들었다'가 아니라 왜 Topic 계약이 필요했고, 왜 WebSocket이 필요했고, 왜 E-Stop과 watchdog이 필요한지 설명해야 한다.

하루 복습 루틴

1. 30초 프로젝트 소개를 소리 내어 말한다.
2. heartbeat -> supervisor -> /fleet/status -> dashboard 흐름을 그린다.
3. Web command -> override -> twist_mux -> cmd_vel 흐름을 그린다.
4. 확인됨/확인 필요/향후 개선을 세 문장으로 분리한다.

반드시 기억	- 면접 답변은 기능 나열이 아니라 문제-구조-검증-한계 순서로 말한다. - 시뮬레이션은 실차가 없을 때 통신 계약과 UI를 검증하기 위한 장치다. - 모르는 것을 숨기지 말고 확인 필요와 다음 검증 계획으로 바꿔 말한다.
면접 한 문장	저는 구현한 범위와 검증이 필요한 범위를 분리해서 설명하고, 다음 실차 검증 계획까지 제시하겠습니다.
주의	암기 문장을 그대로 읽는 느낌보다 본인이 겪은 문제와 연결해서 말해야 한다.

1. 클로봇 공고 분석과 내가 보여줘야 할 역량

왜 필요한가

공고는 서비스 로봇 SW, 응용 모듈, 제어와 센서 연동, 관제 대시보드, UI, 테스트와 배포를 요구한다. 즉 로봇 내부 알고리즘 하나만 보는 것이 아니라 로봇이 현장에서 운영될 때 필요한 통신, 관제, 안전, 검증을 함께 본다.

AIP 프로젝트에서 어떻게 쓰였나

AIP는 ROS2 기반 차량 상태 수집, 웹관제, E-Stop, 수동 override, 비전/thermal alert, Docker sim 실행 구조를 가진다. 신입 포트폴리오에서는 '상용 제품 완성'보다 로봇 응용 SW의 데이터 흐름을 이해하고 정리한 점을 강조한다.

공고 요구	AIP에서 연결할 근거	면접 강조점
서비스 로봇 SW	ROS2 package, launch, msg/srv 구조	로봇을 기능 단위 Node로 나누어 이해
제어/센서 연동	cmd_vel, heartbeat, odom, scan, image topic	Topic 계약과 데이터 흐름 설명
관제 대시보드	FastAPI + WebSocket + Canvas UI	실시간 상태 표시와 명령 전달
테스트/검증	Docker sim, port 분리, 실행 checklist	실차 전 단계의 재현 가능한 검증

면접에서 어떻게 말할까

공고의 업무를 하나씩 프로젝트와 연결해서 말한다. 예를 들어 제어와 센서 연동은 cmd_vel/heartbeat/LiDAR/카메라 Topic으로, 웹/앱 프론트엔드는 FastAPI + WebSocket dashboard로, 테스트와 검증은 Docker sim과 실행 체크리스트로 연결한다.

반드시 기억	- 공고 키워드: ROS2, 센서 연동, 웹관제, 실시간 통신, 테스트/검증. - AIP의 강점은 여러 기술을 하나의 운영 흐름으로 묶은 경험이다. - 신입답게 '구조 이해와 검증 계획'을 강점으로 말한다.
면접 한 문장	클로봇 공고에서 요구하는 웹-로봇 연동, 센서 데이터 처리, 관제 UI 경험을 AIP의 ROS2 Topic 흐름과 대시보드로 설명할 수 있습니다.
주의	상용화 완료, 국책과제 수준 완성처럼 검증하지 않은 성과 표현은 피한다.

2. 프로젝트 한 줄 소개와 30초/1분/3분 답변

왜 필요한가

면접 초반에는 긴 기술 설명보다 프로젝트의 목적과 본인 역할을 짧게 말해야 한다. 한 줄 소개가 명확하면 뒤 질문에서 ROS2, 웹관제, 안전, 실차 확장으로 자연스럽게 이어갈 수 있다.

AIP 프로젝트에서 어떻게 쓰였나

AIP는 산업감시로봇 팀프로젝트를 포트폴리오로 정리한 ROS2 기반 다중 차량 관제/제어 데모다. 현재 확인된 핵심은 3대 시뮬레이션 차량 상태, 지도/위치 표시, 수동 제어와 E-Stop, 센서/alert 흐름의 구조다.

길이	답변 뼈대
30초	ROS2 기반 다중 차량 관제 프로젝트입니다. 저는 서버차량 제어 흐름, 웹관제 연동, 비전/thermal 데이터 흐름, 문서화와 데모 정리를 맡았습니다. 현재는 시뮬레이션 웹관제 검증이 중심입니다.
1분	차량은 heartbeat/odom/scan 같은 Topic으로 상태를 보내고, 중앙 supervisor가 /fleet/status로 모읍니다. dashboard_server가 ROS2 데이터를 WebSocket으로 바꿔 브라우저에 표시하고, E-Stop/override 명령은 다시 ROS2 Topic으로 내려갑니다.
3분	실차가 없는 상황에서도 통신 계약과 UI를 검증하기 위해 Docker sim을 만들고, 실제 차량 확장 시에도 같은 /<ns>/heartbeat, /<ns>/cmd_vel, /<ns>/estop 계약을 유지하는 방향으로 설계했습니다.

면접에서 어떻게 말할까

30초 답변은 목적과 역할만, 1분 답변은 데이터 흐름까지, 3분 답변은 문제-해결-한계-개선 계획까지 말한다. 답변 마지막에는 '현재는 시뮬레이션 검증, 실차 3대는 검증 예정'을 붙여 신뢰도를 높인다.

반드시 기억	- 한 줄: ROS2 기반 다중 로봇 웹관제와 제어 흐름을 정리한 프로젝트. - 30초: 목적, 내 역할, 현재 검증 범위. - 3분: 문제, 구조, 검증, 한계, 다음 계획.
면접 한 문장	AIP는 여러 차량의 상태와 제어 명령을 ROS2 Topic 계약으로 묶고, 웹관제에서 실시간으로 확인/제어하는 프로젝트입니다.
주의	본인이 맡지 않은 메인 AGV 차량 SW까지 직접 구현했다고 말하지 않는다.

3. ROS2가 왜 필요한지: Node, Topic, Message, Service, Action, TF

왜 필요한가

로봇은 모터, 센서, 위치추정, 자율주행, 관제 UI가 동시에 움직인다. ROS2는 이 기능들을 Node로 나누고 Topic/Service/Action/TF 계약으로 연결해 변경과 확장을 쉽게 만든다.

AIP 프로젝트에서 어떻게 쓰였나

AIP에서는 차량별 heartbeat, odom, scan, cmd_vel, estop Topic과 fleet 공통 /fleet/status, /fleet/override, /fleet/alerts Topic을 사용한다. Nav2 goal은 NavigateToPose Action 개념으로 설명하고, 지도와 위치 표시에는 TF와 map/odom/base 계층이 중요하다.

개념	AIP 예시	면접 설명
Node	supervisor_node, dashboard_server, watchdog_node	기능 단위 실행 프로세스
Topic	/<ns>/heartbeat, /fleet/status, /<ns>/cmd_vel	지속적으로 흐르는 상태/명령
Service	costmap clear, save_map_now 계열	짧은 요청-응답
Action	/<ns>/navigate_to_pose	목표와 진행상태가 있는 작업
TF	map -> odom -> base_link	좌표계 관계와 위치 표시

면접에서 어떻게 말할까

ROS2를 '로봇용 통신 프레임워크'라고만 말하지 말고 AIP Topic 예시를 붙인다. Topic은 지속 스트림, Service는 요청-응답, Action은 오래 걸리는 목표 수행, TF는 좌표계 관계라고 짧게 비교한다.

반드시 기억	- Topic: 계속 흐르는 상태/명령 데이터. - Service: 한 번 요청하고 응답받는 작업. - Action: 목표를 보내고 진행/결과를 받는 장시간 작업.
면접 한 문장	AIP에서는 차량 상태는 Topic으로 계속 받고, 목표 이동은 Action 개념으로 보내며, 지도 위 위치 표시는 TF 좌표계가 맞아야 가능합니다.
주의	ROS2를 썼다는 말만 하지 말고 실제 Topic 이름과 Message 역할을 같이 말한다.

4. AIP에서 ROS2가 어떻게 연결되는지

왜 필요한가

다중 차량 관제에서는 차량마다 이름과 Topic이 섞이면 디버깅이 어려워진다. 그래서 차량 namespace와 fleet 공통 Topic을 분리해야 한다.

AIP 프로젝트에서 어떻게 쓰였나

현재 문서와 코드 기준 차량 namespace는 aip1, aip2, aip3이고, fleet 전역 Topic은 /fleet/*를 사용한다. 차량별 Topic은 /aip1/heartbeat처럼 특정 차량에 붙고, /fleet/status는 supervisor가 통합해서 dashboard와 watchdog이 함께 본다.

구분	대표 Topic	역할
차량 상태	/<ns>/heartbeat, /<ns>/odom, /<ns>/scan	개별 차량이 내보내는 생존/위치/센서 정보
차량 제어	/<ns>/override_cmd_vel, /<ns>/estop, /<ns>/cmd_vel	수동/안전/최종 모터 명령 흐름
fleet 상태	/fleet/status, /fleet/peer_poses	중앙 관제와 watchdog이 보는 통합 상태
fleet 이벤트	/fleet/alerts, /fleet/override	센서 경보와 중앙 제어 명령

면접에서 어떻게 말할까

면접에서는 '차량별 Topic과 fleet 공통 Topic을 분리했다'고 먼저 말한다. 그리고 실차/시뮬레이션이 같은 계약을 쓰면 dashboard 입장에서 데이터 출처가 sim인지 real인지보다 Message 계약이 더 중요해진다고 설명한다.

반드시 기억	- 차량 namespace: aip1, aip2, aip3. - fleet 공통 Topic: /fleet/status, /fleet/override, /fleet/alerts. - sim과 real은 같은 계약을 바라보게 만드는 것이 목표.
면접 한 문장	차량별 데이터는 /aipN/*로 분리하고, 관제에서 필요한 통합 데이터는 /fleet/*로 모아 dashboard와 watchdog이 같이 보게 했습니다.
주의	과거 문서의 main/scout_1/scout_2 표현과 최신 aip1/aip2/aip3 표현 차이를 숨기지 않는다.

5. 웹관제가 왜 필요한지: FastAPI, WebSocket, Dashboard

왜 필요한가

로봇을 현장에서 운용하려면 터미널에서 topic echo만 보는 것으로는 부족하다. 운영자는 차량 상태, 지도, 경보, 제어 버튼을 한 화면에서 보고 즉시 판단해야 한다.

AIP 프로젝트에서 어떻게 쓰였나

AIP의 dashboard_server.py는 ROS2 Node이면서 FastAPI 서버 역할을 한다. ROS2 Topic을 구독해 WebSocket JSON이나 image payload로 브라우저에 보내고, 브라우저 명령은 다시 ROS2 Topic/Action 형태로 변환한다. 메인 dashboard는 React/Vue가 아니라 정적 HTML/JavaScript 기반으로 확인된다.

층	기술	역할
ROS2	rclpy node	/fleet/status, /fleet/alerts, /map, /<ns>/cmd_vel 등 publish/subscribe
Backend	FastAPI, WebSocket	ROS2 Message를 JSON/image payload로 변환하고 UI 명령을 ROS2로 전달
Frontend	HTML, CSS, JavaScript, Canvas	차량 카드, 지도, 제어, 경보, 영상 패널 표시

면접에서 어떻게 말할까

웹관제는 '보기 좋은 화면'보다 ROS2와 사용자를 연결하는 운용 계층이라고 말한다. WebSocket을 쓴 이유는 fleet_status, alert, scan, vision 같은 데이터가 계속 변하기 때문이다.

반드시 기억	- FastAPI: ROS2와 브라우저 사이 backend 역할. - WebSocket: 상태와 영상/경보를 실시간 push. - Dashboard: 운영자가 상태를 보고 명령을 내리는 화면.
면접 한 문장	브라우저가 ROS2에 직접 붙는 구조가 아니라, Python backend가 ROS2와 WebSocket 사이에서 bridge 역할을 합니다.
주의	메인 dashboard를 React/Vue로 만들었다고 말하지 않는다. 확인된 구조는 FastAPI + 정적 HTML/JS다.

6. 상태 흐름: heartbeat -> supervisor -> /fleet/status -> dashboard

왜 필요한가

관제의 첫 번째 질문은 '로봇이 살아 있는가'다. heartbeat가 없으면 위치나 명령보다 먼저 통신 단절과 안전 처리를 생각해야 한다.

AIP 프로젝트에서 어떻게 쓰였나

각 차량 또는 sim node는 /<ns>/heartbeat를 보낸다. supervisor_node는 이를 수집해 /fleet/status를 발행하고, dashboard와 watchdog은 /fleet/status를 본다. dashboard는 online/offline, battery/cpu 일부 상태, map marker를 표시한다.

vehicle /<ns>/heartbeat -> supervisor_node -> /fleet/status -> dashboard_server -> browser UI

단계	확인 포인트	실패 시 증상
차량	/<ns>/heartbeat 발행	차량 offline 표시
중앙	supervisor_node 수집	/fleet/status 비어 있음
관제	WebSocket fleet_status push	UI 카드 갱신 안 됨
안전	watchdog timeout 판단	offline E-Stop 검증 필요

면접에서 어떻게 말할까

상태 흐름은 반드시 순서대로 말한다. 차량이 heartbeat를 보내고, supervisor가 fleet 상태로 통합하고, dashboard가 표시하고, watchdog은 offline 조건에서 E-Stop을 낼 수 있다고 연결한다.

반드시 기억	- heartbeat는 차량별 생존 신호다. - /fleet/status는 중앙이 모은 fleet 요약이다. - dashboard와 watchdog은 같은 상태 기반으로 표시/안전 판단을 한다.
면접 한 문장	차량별 heartbeat를 supervisor가 /fleet/status로 통합하고, dashboard와 watchdog이 그 상태를 각각 표시와 안전 판단에 사용합니다.
주의	heartbeat가 있다고 해서 위치추정, 자율주행, 실차 안정성이 모두 검증된 것은 아니다.

7. 제어 흐름: Web command -> override -> twist_mux -> cmd_vel

왜 필요한가

관제 제어는 잘못 연결되면 위험하다. 사용자가 누른 버튼이 어떤 Topic으로 내려가고, 어떤 우선순위로 최종 cmd_vel이 되는지 추적할 수 있어야 한다.

AIP 프로젝트에서 어떻게 쓰였나

AIP에서는 dashboard 명령이 WebSocket으로 backend에 들어오고, backend가 /fleet/override 또는 차량별 override_cmd_vel/estop 계열 Topic을 발행한다. supervisor와 twist_mux가 E-Stop, central override, coordinator, autonomy 입력의 우선순위를 적용해 최종 /<ns>/cmd_vel로 만든다.

```
Browser control -> WebSocket command -> dashboard_server -> /fleet/override or /<ns>/override_cmd_vel -> supervisor/twist_mux -> /<ns>/cmd_vel
```

입력	우선순위 의미	설명
E-Stop/lock	가장 높음	위험 상황에서 모든 motion 차단
central override	높음	관제자가 직접 수동 조작
coord_cmd_vel	중간	follower/협조 주행 명령
autonomy_cmd_vel	낮음	Nav2 등 자율주행 출력

면접에서 어떻게 말할까

면접에서는 '수동 제어가 Nav2보다 우선하고, E-Stop은 더 높은 우선순위로 motion을 막아야 한다'고 설명한다. cmd_vel은 보통 linear.x와 angular.z 중심으로 차동구동 차량에 전달된다고 덧붙인다.

반드시 기억	- UI 명령은 WebSocket으로 backend에 들어온다. - ROS2 제어 Topic은 override/estop/cmd_vel로 나뉜다. - twist_mux는 E-Stop과 수동 override 우선순위를 보장하기 위한 구조다.
면접 한 문장	사용자 명령은 WebSocket으로 들어와 ROS2 override Topic이 되고, twist_mux 우선순위를 거쳐 최종 cmd_vel로 차량에 전달됩니다.
주의	E-Stop end-to-end 실차 정지 검증은 차량별로 확인 필요라고 분리해서 말한다.

8. 시뮬레이션이 왜 필요한지: 실차 없는 상황의 검증 전략

왜 필요한가

실차가 없거나 수업용 포트와 충돌하는 상황에서도 관제 UI, ROS2 Message 계약, supervisor/watchdog 흐름은 검증해야 한다. 시뮬레이션은 안전하고 반복 가능한 통합 테스트 환경이다.

AIP 프로젝트에서 어떻게 쓰였나

AIP sim은 docker/sim compose로 실행하고 host 포트는 dashboard 18080, Foxglove 18765를 사용한다. 컨테이너 내부 dashboard는 8080을 유지하고, 외부 접속 포트만 바꾸어 수업용 Spark 8080과 충돌하지 않게 했다. demo_patrol_node는 sim 전용 자동 움직임을 만든다.

구분	시뮬레이션	실차
목적	통신/UI/상태 흐름 사전 검증	센서/모터/네트워크 포함 실제 동작 검증
주소	http://localhost:18080, ws://localhost:18765	현장 네트워크와 ROS_DOMAIN_ID 설정 필요
자동 이동	demo_patrol_node로 그럴듯한 움직임 생성	Nav2/driver/센서 기반 검증 필요
표현	확인됨: sim 관제	확인 필요: 실차 3대 E2E

면접에서 어떻게 말할까

면접에서는 시뮬레이션을 '실차를 대신해 과장하는 도구'가 아니라 '실차 전 통신과 관제 흐름을 검증하는 단계'라고 말한다. sim과 real을 분리해 둔 점도 중요하다.

반드시 기억	- 시뮬레이션은 실차 검증의 대체가 아니라 사전 검증이다. - 현재 접속 주소: http://localhost:18080. - demo_patrol_node는 sim 전용이며 실제 차량 launch에는 넣지 않는다.
면접 한 문장	실차가 없는 상황에서도 ROS2 Topic 계약과 웹관제 흐름을 반복 검증하기 위해 Docker sim을 사용했습니다.
주의	시뮬레이션에서 움직인 것을 실차 자율주행 검증 완료로 말하지 않는다.

9. 실제 차량 확장 전략: aip1, TurtleBot3, ESP32 커스텀 차량

왜 필요한가

클로봇 업무는 이기종 로봇 관제와 연결될 가능성이 크다. 서로 다른 차량을 같은 화면에서 보려면 차량 내부 하드웨어가 달라도 외부 ROS2 인터페이스는 공통이어야 한다.

AIP 프로젝트에서 어떻게 쓰였나

AIP는 aip1 메인 차량, aip2 TurtleBot3, aip3 custom vehicle 방향으로 정리되어 있다. 공통 계약은 `/<ns>/heartbeat`, `/<ns>/cmd_vel`, `/<ns>/override_cmd_vel`, `/<ns>/estop`, `/<ns>/odom`, `/<ns>/scan`이다. aip3 custom driver와 실차 전체 검증은 확인 필요로 둔다.

차량	현재 방향	면접 표현
aip1	메인 AGV, LiDAR/SLAM/Nav2/serial_bridge 기반 문서 존재	메인 차량 연동 구조는 문서와 launch 기준으로 설명
aip2	TurtleBot3 Burger, bringup/SLAM/Nav2 방향	표준 ROS2 로봇을 fleet 계약에 붙이는 확장 예
aip3	ESP32/커스텀 차량, placeholder와 확인 필요 항목 존재	수동/heartbeat부터 단계적으로 검증 예정

면접에서 어떻게 말할까

면접에서는 '실차 3대 모두 완성'보다 '이기종 차량을 공통 Topic 계약으로 묶는 전략'을 말한다. TurtleBot3는 표준 ROS2 bringup이 있고, ESP32 커스텀 차량은 중앙 PC 또는 SBC가 더 많은 계산을 맡는 구조가 현실적이라고 설명한다.

반드시 기억	- 이기종 차량의 핵심은 공통 Topic 계약이다. - aip1/aip2/aip3는 namespace로 구분한다. - custom vehicle driver 완성도와 실차 E2E는 확인 필요하다.
면접 한 문장	하드웨어가 달라도 <code>/<ns>/heartbeat</code> 와 <code>/<ns>/cmd_vel</code> 같은 공통 인터페이스를 맞추면 관제와 supervisor는 같은 방식으로 다룰 수 있습니다.
주의	aip3 STS3215/ESP32 차량을 완성된 상용 드라이버처럼 말하지 않는다.

10. ESP32/Pi 없는 상황에서 가능한 것과 어려운 것

왜 필요한가

ESP32는 모터, 간단한 센서, Wi-Fi 통신에는 적합하지만 SLAM/Nav2 같은 무거운 연산을 단독으로 맡기기 어렵다. 그래서 가능한 범위와 어려운 범위를 나누어 설계해야 한다.

AIP 프로젝트에서 어떻게 쓰였나

현재 보유 차량은 ESP32 모터, ESP32-S3 LiDAR, ESP32_CAM 카메라 구성으로 볼 수 있다. Pi가 없으면 중앙 PC가 ROS2 brain 역할을 맡고 ESP32는 motor/scan/camera gateway 역할부터 시작하는 것이 현실적이다. micro-ROS 또는 UDP/WebSocket bridge는 후보지만 실제 firmware 계약은 확인 필요하다.

가능한 것	어려운 것	첫 검증 목표
heartbeat, battery/status 송신	ESP32 단독 SLAM/Nav2	/aip3/heartbeat online 표시
manual cmd_vel -> motor PWM	복잡한 TF/map/localization	WASD/override로 바퀴 반응
ESP32_CAM MJPEG/HTTP stream	고성능 YOLO 실시간 추론	dashboard 영상 표시
ESP32-S3 sensor gateway	장시간 네트워크 안정성 보장	scan/거리 데이터 형식 확인

면접에서 어떻게 말할까

면접에서는 'ESP32만으로 완전 자율주행'이 아니라 'ESP32는 실시간 I/O, 중앙 PC는 ROS2/관제/경로계획'이라고 역할을 나누어 말한다. 첫 목표는 heartbeat, manual cmd_vel, E-Stop, camera stream 검증이다.

반드시 기억	- ESP32: 모터 I/O, 간단 센서, 상태 송신에 적합. - 중앙 PC/SBC: ROS2, SLAM/Nav2, 관제, 데이터 통합에 적합. - Pi 없이 시작할 때는 수동 제어와 안전부터 검증한다.
면접 한 문장	Pi가 없을 때는 중앙 PC를 계산 주체로 두고, ESP32 차량은 공통 Topic 계약을 만족하는 I/O 노드로 단계적으로 붙이는 전략이 현실적입니다.
주의	ESP32 단독으로 SLAM/Nav2와 안정적 다중 로봇 자율주행이 가능하다고 말하지 않는다.

11. LiDAR, odom, TF, map이 왜 중요한지

왜 필요한가

로봇이 지도 위에서 움직이려면 센서값만으로는 부족하다. LiDAR는 주변 장애물을 보고, odom은 이동량을 추정하고, TF는 좌표계를 연결하고, map은 위치를 해석할 기준을 제공한다.

AIP 프로젝트에서 어떻게 쓰였나

AIP dashboard는 지도, 차량 pose, scan, peer pose를 표시한다. 실차에서는 `/<ns>/scan`, `/<ns>/odom`, `map -> odom -> base_link` TF가 안정적으로 맞아야 Nav2나 관제 위치 표시가 의미를 가진다. 시뮬레이션에서는 이 데이터를 가짜로 만들어 UI와 흐름을 검증한다.

데이터	대표 Topic/Frame	없거나 틀리면
LiDAR	<code>/<ns>/scan</code>	장애물 인식/SLAM 품질 저하
Odometry	<code>/<ns>/odom</code>	움직였는지 추적 어려움
TF	<code>map -> odom -> base_link</code>	지도 위 pose 계산 실패
Map	<code>/map</code> , <code>/map_static</code>	위치와 경로의 기준 없음

면접에서 어떻게 말할까

면접에서는 LiDAR를 '장애물 보는 센서'로 끝내지 말고 odom/TF/map과 연결한다. 지도 위 마커가 정확하려면 좌표계가 맞아야 하고, `cmd_vel`로 움직여도 odom이 돌아오지 않으면 관제는 실제 이동을 알 수 없다고 설명한다.

반드시 기억	- LiDAR: 주변 거리/장애물. - odom: 로봇의 이동량 추정. - TF/map: 좌표계와 지도 기준.
면접 한 문장	지도 위 차량 위치가 맞으려면 scan, odom, TF, map이 함께 맞아야 하며, 하나라도 틀리면 관제 표시는 신뢰하기 어렵습니다.
주의	dashboard에 마커가 보인다고 실제 localization이 완성됐다고 말하지 않는다.

12. 카메라, thermal, perception alert가 왜 필요한지

왜 필요한가

산업감시로봇은 이동만 하는 로봇이 아니라 현장 상태를 보는 로봇이다. 카메라와 thermal 데이터는 작업자가 원격으로 상황을 판단하고 경보를 받을 수 있게 한다.

AIP 프로젝트에서 어떻게 쓰였나

AIP에는 Vision Pi HTTP bridge, ROS2 image topic, thermal driver, patrol_monitor_node, central_fusion_node, /fleet/alerts 구조가 문서와 코드에 있다. RGB/thermal frame을 dashboard에 표시하고 threshold 기반 alert를 만들 수 있는 흐름을 정리했다. YOLOv8 기반 fire/smoke 검출은 코드 경로가 있어도 모델/정확도 검증은 확인 필요하다.

흐름	AIP 요소	면접 표현
입력	RGB camera, ESP32_CAM/HTTP, thermal frame	센서 입력을 ROS2로 가져오는 단계
처리	OpenCV, thermal threshold, fusion node	프레임 변환과 경보 조건 계산
출력	/fleet/alerts, /fleet/perception_viz/ <vehicle>	관제 화면에 표시할 이벤트와 이미지
검증 필요	YOLO model/accuracy, latency/FPS	수치와 모델은 추가 검증 예정

면접에서 어떻게 말할까

면접에서는 영상 처리를 'AI로 다 했다'가 아니라 image 입력, OpenCV 변환, thermal threshold, alert 발행, dashboard 표시의 단계로 말한다. AI 모델은 선택 기능이며 검증 범위를 분리한다.

반드시 기억	- 카메라: 원격 확인과 시각 증거. - thermal: 온도 기반 이상 징후. - PerceptionAlert: 센서 판단을 관제 이벤트로 바꾸는 메시지.
면접 한 문장	카메라와 thermal 데이터는 단순 표시가 아니라 /fleet/alerts로 관제 이벤트가 되어 운영자가 판단할 수 있게 만드는 흐름입니다.
주의	YOLO 정확도, 실제 화재/연기 검출 성능은 확인 필요라고 말한다.

13. E-Stop, watchdog, command timeout이 왜 중요한지

왜 필요한가

로봇 제어에서 안전은 기능이 아니라 기본 조건이다. 통신이 끊기거나 UI 명령이 오래 남아 있거나 차량이 offline이면 로봇이 계속 움직이지 않도록 막아야 한다.

AIP 프로젝트에서 어떻게 쓰였나

AIP에는 watchdog_node가 /fleet/status를 보고 offline 차량에 E-Stop을 발행하는 구조, supervisor가 /fleet/override를 차량별 estop/zero Twist로 route하는 구조, twist_mux lock 우선순위 구조가 있다. ESP32 firmware에도 command watchdog 개념이 문서화 되어 있지만 차량별 end-to-end 검증은 확인 필요하다.

안전 장치	역할	확인/주의
E-Stop	수동 또는 watchdog 기반 정지 명령	차량별 실제 정지 검증 필요
watchdog	heartbeat timeout 감시	offline 판단 기준 설명 가능해야 함
twist_mux lock	cmd_vel 우선순위 차단	estop lock 연결 확인 필요
firmware timeout	명령 끊김 시 PWM 0	ESP32 차량 구현 범위 확인 필요

면접에서 어떻게 말할까

면접에서는 안전을 UI 버튼으로만 설명하지 않는다. E-Stop 명령 경로, watchdog 조건, stale command timeout, motor PWM zero 같은 레벨을 나누어 말한다.

반드시 기억	- E-Stop은 최상위 motion 차단 명령이다. - watchdog은 heartbeat/offline을 감시한다. - command timeout은 오래된 명령이 계속 적용되는 것을 막는다.
면접 한 문장	안전 구조는 dashboard 버튼, supervisor routing, twist_mux lock, 차량 firmware watchdog이 단계별로 함께 맞아야 완성됩니다.
주의	모든 차량에서 실제 정지가 검증됐다고 말하지 말고, end-to-end 검증 예정이라고 분리한다.

14. Docker, 포트, 실행 환경을 왜 분리했는지

왜 필요한가

면접관이나 팀원이 데모를 재현하려면 실행 환경이 명확해야 한다. 특히 수업용 Docker stack이 8080, 5000, 3306, 7077, 8888, 4040, 8001을 쓰는 상황에서는 포트 충돌을 피해야 한다.

AIP 프로젝트에서 어떻게 쓰였나

AIP sim은 host dashboard 포트를 18080으로, Foxglove host 포트를 18765로 바꾸었다. 컨테이너 내부 서비스는 그대로 8080/8765를 사용하므로 코드 내부 주소와 외부 접속 주소를 분리했다. 이는 수업용 Spark 8080과 충돌하지 않게 하기 위한 결정이다.

서비스	container 내부	host 접속	이유
AIP dashboard	8080	18080	수업용 Spark 8080과 충돌 방지
Foxglove bridge	8765	18765	기존 8765 사용 가능성과 충돌 예방
수업용 Spark	8080	8080	건드리지 않음
수업용 MLflow	5000	5000	건드리지 않음

면접에서 어떻게 말할까

면접에서는 Docker를 '편해서 썼다'보다 재현성, 의존성 고정, 포트 충돌 관리, sim/real 분리를 위해 사용했다고 말한다. 내부 포트와 host 포트의 차이도 설명하면 좋다.

반드시 기억	- host 포트: 사용자가 접속하는 외부 포트. - container 포트: 컨테이너 내부 서비스 포트. - AIP 웹관제 주소는 <code>http://localhost:18080</code>.
면접 한 문장	수업용 포트와 충돌하지 않도록 AIP sim의 host 포트만 18080/18765로 바꾸고, 컨테이너 내부 포트는 유지했습니다.
주의	실차 central stack과 sim stack을 섞어 실행했다고 말하지 않는다.

15. 프로젝트에서 겪은 문제와 해결 과정

왜 필요한가

면접은 성공 결과만 듣는 자리가 아니다. 어떤 문제가 있었고 어떻게 원인을 좁혔으며 무엇을 배웠는지가 신입에게 중요한 평가 요소다.

AIP 프로젝트에서 어떻게 쓰였나

AIP에서 정리된 문제는 포트 충돌, sim build typo, 시뮬레이션이 멈춰 보이는 문제, sim/real 구분, namespace 문서 차이, ESP32-only 차량의 한계, E-Stop end-to-end 검증 필요 등이다. 이 문제들은 실제 로봇 SW에서 자주 나오는 통신/환경/검증 문제와 연결된다.

문제	해결/대응	배운 점
수업용 포트 충돌	AIP host 포트를 18080/18765로 변경	실행 환경 분리의 중요성
sim build typo	uvicorn, return, warning 오타 수정 계획 반영	재빌드 로그 확인 필요
sim/real 혼동	demo_patrol_node를 sim 전용으로 명시	검증 범위 표현의 중요성
ESP32-only 한계	중앙 PC brain + ESP32 I/O 전략	하드웨어 역할 분리

면접에서 어떻게 말할까

문제 답변은 '문제 -> 원인 가설 -> 확인 방법 -> 조치 -> 남은 검증' 순서가 좋다. 예를 들어 포트 충돌은 수업용 stack 포트를 보존하고 AIP host 포트만 바꾼 결정으로 설명한다.

반드시 기억	- 좋은 문제 답변은 원인과 확인 방법이 있다. - 해결 후 남은 리스크를 같이 말하면 신뢰도가 오른다. - 로봇 프로젝트에서는 환경/네트워크/센서 문제가 기능만큼 중요하다.
면접 한 문장	문제를 숨기기보다 원인 가설을 세우고 로그, Topic, 포트, UI 상태로 좁혀 해결한 경험을 말하겠습니다.
주의	모든 문제를 완전히 해결했다고 묶어 말하지 말고 남은 검증을 같이 제시한다.

16. 과장 없이 말해야 하는 한계와 개선 계획

왜 필요한가

지원용 포트폴리오에서 가장 위험한 것은 구현하지 않은 것을 구현한 것처럼 말하는 것이다. 신입에게는 '모르는 것을 구조화하고 다음 실험을 설계하는 태도'도 중요한 역량이다.

AIP 프로젝트에서 어떻게 쓰였나

현재 확인된 범위는 시뮬레이션 웹관제, ROS2 Topic 계약, supervisor/dashboard/watchdog 구조, 일부 센서/비전 경로의 코드와 문서다. 확인 필요 범위는 실차 3대 완전 자율주행, custom ESP32 차량 driver 완성, 차량별 E-Stop end-to-end, YOLO 성능, latency/FPS 수치다.

분류	내용	면접 표현
확인됨	웹관제, Docker sim, /fleet/status 흐름	현재 데모에서 확인 가능한 범위
확인 필요	실차 3대 자율주행, E-Stop E2E, ESP32 driver	다음 검증 항목으로 제시
향후 개선	latency/FPS 측정, 로그 감사, reconnect/stale 처리	운영 안정성 개선 계획
금지 표현	완전 자율주행, 상용화 완료, 실차 검증 완료	근거 없으면 사용 금지

면접에서 어떻게 말할까

면접에서는 한계를 먼저 방어적으로 말하지 말고, 확인된 것과 다음 검증 계획을 같이 말한다. '아직 실차 3대는 검증 전이지만, 공통 Topic 계약을 맞추고 heartbeat/manual cmd_vel/E-Stop부터 단계적으로 검증할 계획'이라고 말한다.

반드시 기억	- 확인됨: sim 관제와 통신 흐름. - 확인 필요: 실차 3대 E2E, custom driver, safety 검증. - 개선 계획: heartbeat -> manual -> E-Stop -> odom/scan -> Nav2 순서.
면접 한 문장	현재는 시뮬레이션과 관제 흐름 검증이 중심이고, 실차는 공통 Topic 계약을 기준으로 단계 검증할 계획입니다.
주의	실차 검증 전에는 자율주행 가능, 군집 주행 완료, 상용 수준 안정화 같은 표현을 쓰지 않는다.

17-A. 예상 면접 질문 1-20과 답변 키워드

No	예상 질문	답변 키워드
1	AIP 프로젝트를 30초로 소개해 주세요.	ROS2 기반 다중 차량 관제, 웹관제, 상태/제어 흐름, 현재 sim 검증
2	본인이 맡은 역할은 무엇인가요?	서브차량 제어 흐름, dashboard 연동, 비전/thermal 흐름, 문서화/데모 정리
3	ROS2를 왜 사용했나요?	Node 분리, Topic 계약, 이기종 차량 확장, real/sim 공통 인터페이스
4	Topic과 Service, Action 차이는?	Topic stream, Service request-response, Action long-running goal
5	/heartbeat와 /fleet/status 차이는?	차량별 생존 신호 vs supervisor가 모은 fleet 요약
6	namespace를 왜 나누나요?	차량별 topic 충돌 방지, aip1/aip2/aip3 구분
7	dashboard는 ROS2와 어떻게 연결되나요?	dashboard_server.py가 ROS2 node + FastAPI/WebSocket bridge
8	WebSocket을 쓴 이유는?	상태/alert/image/scan이 계속 바뀌므로 push 기반 실시간 갱신
9	rosbridge를 사용했나요?	현재 메인 dashboard는 FastAPI backend bridge, rosbridge 사용은 확인되지 않음
10	상태 데이터 흐름을 설명해 주세요.	vehicle heartbeat -> supervisor -> /fleet/status -> dashboard/watchdog
11	제어 데이터 흐름을 설명해 주세요.	UI -> WebSocket -> backend -> override/estop -> twist_mux -> cmd_vel
12	cmd_vel은 무엇인가요?	geometry_msgs/Twist, linear.x/angular.z 중심의 이동 명령
13	twist_mux가 왜 필요한가요?	E-Stop, manual, coord, autonomy 우선순위 정리
14	E-Stop은 어떻게 처리되나요?	dashboard/watchdog -> /fleet/override or estop -> supervisor -> lock/zero Twist
15	watchdog의 역할은?	heartbeat/offline 감시, timeout 시 safety command 발행
16	시뮬레이션을 왜 만들었나요?	실차 없이 통신 계약, UI, 상태/제어 흐름 반복 검증
17	sim과 real은 구분되어 있나요?	demo_patrol_node는 sim 전용, real launch는 별도, 공통 Topic 계약 유지 목표
18	포트를 왜 18080으로 바꿨나요?	수업용 Spark 8080 보존, AIP host port만 변경
19	Foxglove 주소는?	ws://localhost:18765, 충돌 예방을 위한 host port 변경
20	현재 데모에서 확인 가능한 것은?	CONNECTED, MAP READY, 3 online, sim vehicle motion, dashboard display
반드시 기억		- 질문에는 Topic 이름 하나 이상을 붙여 답한다. - 현재 확인된 데모 범위와 실차 검증 범위를 분리한다. - 모르면 확인 필요와 검증 계획으로 답변을 달는다.
면접 한 문장		AIP는 sim과 real이 같은 ROS2 계약을 바라보게 만들어 관제와 안전 흐름을 먼저 검증한 프로젝트입니다.
주의		질문 답변에서 '다 했습니다'보다 '여기까지 확인했고 다음은 이렇게 검증합니다'가 더 안전하다.

17-B. 예상 면접 질문 21-40과 답변 키워드

No	예상 질문	답변 키워드
21	실제 차량 3대 운용은 가능한가요?	구조는 준비, 실차 3대 E2E는 확인 필요, 단계 검증 계획
22	ESP32 차량만으로 가능한가요?	I/O 가능, SLAM/Nav2 단독은 어려움, 중앙 PC brain 전략
23	TurtleBot3를 붙이면 무엇을 확인하나요?	heartbeat, cmd_vel, odom, scan, TF, Nav2 bringup
24	LiDAR가 왜 필요한가요?	장애물/SLAM/localization, map과 연결
25	odom과 TF가 왜 중요한가요?	지도 위 pose, Nav2, dashboard marker 신뢰성
26	map_ready는 무엇인가요?	관제가 map 표시와 주행 판단에 쓰는 준비 상태 신호
27	카메라 데이터 흐름은?	camera/HTTP -> ROS2 image -> fusion/visualization -> dashboard
28	thermal alert는 어떻게 만들 수 있나요?	thermal max/hotspot threshold -> PerceptionAlert -> /fleet/alerts
29	YOLO 기능은 검증됐나요?	코드 경로는 있으나 모델/정확도/현장 검증은 확인 필요
30	OpenCV는 어디에 쓰였나요?	decode/resize/overlay/encode, thermal visualization
31	Docker를 왜 썼나요?	재현성, 의존성, 포트 분리, sim 실행 간소화
32	수업용 Docker stack과 충돌하지 않나요?	수업 포트 보존, AIP host 18080/18765 사용
33	가장 어려웠던 문제는?	sim/real 구분, 포트 충돌, E-Stop E2E 검증 범위 정리 중 하나 선택
34	문서와 코드가 다르면 어떻게 하나요?	코드/launch/msg 기준 확인, 문서는 확인 필요로 갱신
35	본인 역할과 팀 역할은 어떻게 구분하나요?	본인: 제어 흐름/웹관제/비전 연동/문서화, 팀: 메인 차량 SW 등 분리
36	왜 클로봇 직무와 맞나요?	이기종 로봇 관제, ROS2, 센서 연동, UI, 테스트/검증 경험
37	입사 후 무엇을 보완하고 싶나요?	실차 bringup, Nav2/TF, safety test, WebSocket schema, logs
38	이 프로젝트의 한계는?	실차 3대 자율주행 미검증, ESP32 driver, YOLO 성능, latency 수치
39	다음 실험 순서는?	heartbeat -> manual cmd_vel -> E-Stop -> odom/scan/TF -> map/Nav2 -> multi robot
40	마지막으로 강조할 강점은?	과장 없이 구조화하고 검증 계획을 세우는 로봇 응용 SW 태도

반드시 기억	- 하드웨어 질문은 가능한 것과 어려운 것을 분리한다. - 센서 질문은 scan/odom/TF/map 흐름으로 연결한다. - 직무 적합성은 관제, 제어, 센서, 검증을 묶어 말한다.
면접 한 문장	제 강점은 로봇의 상태, 제어, 센서 데이터를 하나의 운용 흐름으로 연결하고 검증 범위를 솔직하게 관리한 경험입니다.
주의	팀 프로젝트에서 다른 팀원이 맡은 차량 SW를 본인 단독 성과로 말하지 않는다.

18. 마지막 암기 카드: 꼭 외울 문장, 금지 표현, 체크리스트

꼭 외울 문장

1. AIP는 여러 차량의 상태와 제어 명령을 ROS2 Topic 계약으로 묶고 웹관제로 확인하는 프로젝트입니다.
2. 차량별 데이터는 /aipN/*로 분리하고, 통합 상태와 명령은 /fleet/*로 모읍니다.
3. heartbeat는 차량별 생존 신호이고 /fleet/status는 supervisor가 만든 fleet 요약입니다.
4. WebSocket은 계속 변하는 상태, alert, image 데이터를 브라우저에 push하기 위해 사용했습니다.
5. E-Stop은 dashboard 버튼만이 아니라 supervisor, twist_mux, firmware watchdog까지 이어져야 완성됩니다.
6. 시뮬레이션은 실차를 과장하기 위한 것이 아니라 실차 전 통신과 관제 흐름을 검증하기 위한 단계입니다.
7. ESP32는 모터/센서 I/O에 적합하고, SLAM/Nav2 같은 계산은 중앙 PC나 SBC가 맡는 구조가 현실적입니다.
8. 현재는 sim 관제 검증이 중심이고 실차 3대 완전 자율주행은 확인 필요입니다.

금지 표현	대신 이렇게 말하기
실차 3대 자율주행 완료	실차 3대 E2E는 검증 예정이며, sim과 공통 Topic 계약을 먼저 검증했습니다.
YOLO 화재 감지 완성	YOLO 경로는 있으나 모델/정확도는 확인 필요이고 thermal threshold alert는 설명 가능합니다.
ESP32만으로 Nav2 가능	ESP32는 I/O, 중앙 PC/SBC는 ROS2 계산을 맡는 구조가 현실적입니다.
제가 메인 AGV 전체를 만들었습니다	팀 구현과 제 역할을 분리해서 설명하겠습니다.
반드시 기억	- 외울 문장은 Topic 이름과 함께 말한다. - 금지 표현을 피하는 것이 신뢰도다. - 마지막에는 다음 검증 계획을 말한다.
면접 한 문장	확인된 것은 자신 있게, 확인이 필요한 것은 다음 검증 계획과 함께 솔직하게 말하겠습니다.
주의	암기 문장만 반복하지 말고 실제 dashboard와 sim 실행 화면을 떠올리며 말한다.

19. 면접 직전 점검표

실행 확인

면접 전날 또는 당일에는 Docker sim이 켜지는지, <http://localhost:18080>에서 dashboard가 뜨는지, 차량 3대 online과 map ready가 보이는지 확인한다. Foxglove는 <ws://localhost:18765>로 접속하는 기준을 기억한다.

체크	질문	내 답변 기준
프로젝트 소개	30초로 말할 수 있나?	목적, 역할, 현재 검증 범위
상태 흐름	heartbeat부터 dashboard까지 그릴 수 있나?	heartbeat -> supervisor -> /fleet/status -> WebSocket
제어 흐름	cmd_vel까지 그릴 수 있나?	UI -> override -> twist_mux -> /cmd_vel
실차 확장	ESP32/TB3/aip1 차이를 말할 수 있나?	공통 Topic 계약과 확인 필요 분리
한계	과장 질문에 버틸 수 있나?	확인됨/확인 필요/향후 개선
직무 연결	클로봇 공고와 연결되나?	ROS2, 웹관제, 센서, 안전, 검증

마지막 답변 프레임

저는 이 프로젝트를 통해 로봇 응용 SW가 단일 기능 구현보다 통신 계약, 상태 감시, 안전 제어, UI, 검증 환경을 함께 맞추는 작업이라는 점을 배웠습니다. 아직 실차 3대 전체 검증은 남아 있지만, 확인된 범위를 기준으로 단계적으로 확장하겠습니다.

반드시 기억	- 면접 직전에는 30초 소개와 두 개의 흐름도만 반복한다. - 실행 주소: http://localhost:18080, ws://localhost:18765. - 합격 전략: 구조를 설명하고, 한계를 숨기지 않고, 다음 검증을 제시한다.
면접 한 문장	로봇 응용 SW 개발자로서 관제, 제어, 센서, 안전, 검증을 하나의 흐름으로 이해하고 성장하겠습니다.
주의	마지막 답변에서 자신감과 솔직함의 균형을 잃지 않는다.